Al-Maaref University - Faculty of Sciences

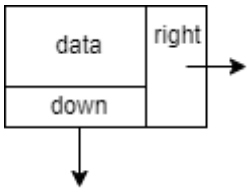
CSC 320 Data Structures - Fall 2023

Programming Project

In this project, you are required to implement a matrix as a single linked list along with some methods to manipulate it.

Every element of the matrix is represented in a **`MatrixNode`** that has a variable of type integer called **`data`**, and two references to **`MatrixNodes`** called **`right`** and **`down`** as illustrated in the below figure.

The code of **`MatrixNode`** starts as follows:



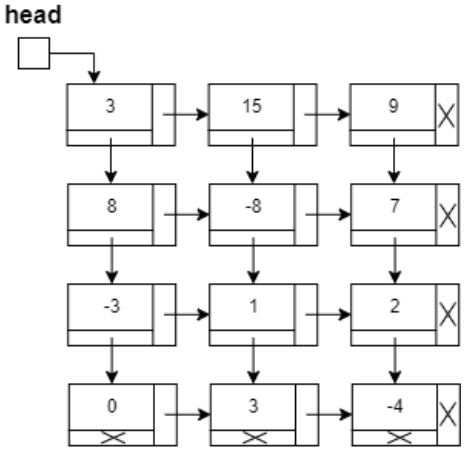

```
private static class MatrixNode {
      int data;
      MatrixNode right, down;
      …
}
```

The class **`MatrixLinkedList`** contains a reference to the top left node of the matrix called **`head`**, along with two integers to save the number of **`rows`** and number of **`columns`**. The code of **`MatrixLinkedList`** starts as follows:

```
public class MatrixLinkedList {
      private MatrixNode head;
      private int rows, columns;

      // MatrixNode class inserted here.
      …
}
```

Here is an illustration of an example matrix as described above. This matrix contains 4 rows and 3 columns.



Complete the implementation of

`MatrixLinkedList` by providing the code of the following methods, and then test them in a main method. The required methods are:

- **`void createMatrix (ArrayList<Integer>[] matrixData)`** Fills in the matrix from an array of ArrayLists. *Assume that the given Arraylists have equal number of elements.*
- **`ArrayList<Integer> getRow(int rowIndex)`** Returns the data of a specific row.
- **`ArrayList<Integer> getColumn(int colIndex)`** Returns the data of a specific column.
- **`int rowSum(int rowIndex)`** Calculates the sum of a row.
- **`double columnAverage(int colIndex)`** Calculates the average of a column.
- **`boolean find(int data)`** Finds a data in the matrix.
- **`void replaceData(int rowIndex, int colIndex, int newData)`** Replaces the data at a position (row and column).
- **`String toString()`** Returns a string representation of the matrix formatted in a grid.
- **`void scalarMultiply(int scalar)`** Multiplies the entire matrix by a scalar.
- <u>Bonus Task</u>: **`void add(MatrixLinkedList matrix)`** Adds a matrix to the current matrix.

<u>Notes:</u>

- At least one of your methods should be **recursive**.
- To implement the above methods you **may** need to implement some private helper methods such as:
  - **`MatrixNode getRowNode (int rowIndex)`**
  - **`MatrixNode getColumnNode (int colIndex)`**
  - **`AddFirstNode(int data)`**
  - **`AddRowAfter(MatrixNode node)`**
  - *and any other methods that you find useful …*
- Add any necessary constructors and/or functions in **`MatrixNode`**.
- Make sure you give meaningful names to your local variables.
- Make sure you implement your methods in the most efficient way.
- Always make sure that the matrix is not empty. If it is empty, throw **`NoSuchElementException`**.
- Make sure that the input values **`rowIndex`** and **`colIndex`** are valid. If they are not valid, throw **`IndexOutOfBoundsException`**.